

Avaliação 2 - 7,0 pts

- 1) (1,0 pt) Implemente uma função que recebe uma lista de pontos no plano cartesiano e calcula a distância Euclidiana bidimensional de cada ponto à origem.

Obs.:

Para pontos bidimensionais, $P = (p_x, p_y)$ e $Q = (q_x, q_y)$, a distância é computada como:

$$\sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}.$$

- 2) (1,5 pts) Crie uma função em Haskell que recebe uma lista de elementos e contabilize a quantidade de elementos iguais consecutivos dessa lista.

Por exemplos:

```
> consecutivo "aaaabccaadeeee"
> [(4, 'a'), (1, 'b'), (2, 'c'), (2, 'a'), (1, 'd'), (4, 'e')]
> consecutivo [1, 1, 2, 3, 3, 4, 4, 4, 5]
> [(2,1), (1, 2), (2, 3), (3, 4), (1, 5)]
```

- 3) (2,0 pts) A função `remdups` remove elementos iguais adjacentes de uma lista, conservando só um dos elementos. Por exemplo, `remdups [1,2,2,3,3,3,1,1] = [1,2,3,1]`.

Defina `remdups` usando `foldr`.

- 4) (2,5 pts) Escreva uma função **part k n** que recebe dois inteiros e retorna uma lista com todas as listas `p` de inteiros que satisfazem as condições:

- a soma dos elementos de `p` dá `n`;
- a lista `p` só possui elementos positivos (não tem zeros nem negativos);
- a lista `p` está em ordem não-decrescente;
- a lista `p` possui apenas elementos maiores ou iguais a `k`.

A lista retornada não pode ter listas repetidas, mas a ordem em que aparecem não importa.

Exemplos:

```
> part 3 5
[[5]]
> part 1 4
[[1,1,1,1],[1,1,2],[1,3],[2,2],[4]]
> part 2 5
[[2,3],[5]]
> part (-1) 3
[[1,1,1],[1,2],[3]]
```